

JUICE-SHOP

PENETRATION TEST REPORT

Comprehensive penetration testing of the
OWASP Juice-Shop vulnerable web application.



DATE
25.06.2026



TARGET
OWASP Juice-Shop



ASSESSMENT
Black-box Penetration Test



FINDINGS
14



CLASS
Confidential

Project Overview

OWASP Juice Shop — это специально уязвимое веб-приложение для обучения веб-безопасности и тестирования на проникновение. Проект поддерживается OWASP и содержит десятки типовых уязвимостей (SQL Injection, XSS, Authentication, IDOR и др.).

Цель: провести полный цикл компрометации веб-приложения, выявить критические уязвимости из OWASP Top 10 и продемонстрировать возможность получения административного доступа.

Период тестирования: 25.06.2026

Методология: OSINT, сканирование портов (nmap), фаззинг (gobuster, XSSStrike), ручной анализ (Burp Suite, curl), эксплуатация (SQLi, XSS, JWT-криптоанализ, подмена сессий).

Project Scope

ОПЛАТФОРМА: Web Application (OWASP Juice Shop)

СРЕДА: Docker (localhost:3000)

ТЕХНОЛОГИЧЕСКИЙ СТЕК:

- Frontend: TypeScript, Angular, HTML5, CSS/SCSS, Bootstrap, Angular Material
- Backend: Node.js, Express.js, TypeScript/JavaScript
- Database: SQLite
- Authentication: JWT (JSON Web Token), bcrypt для хеширования паролей

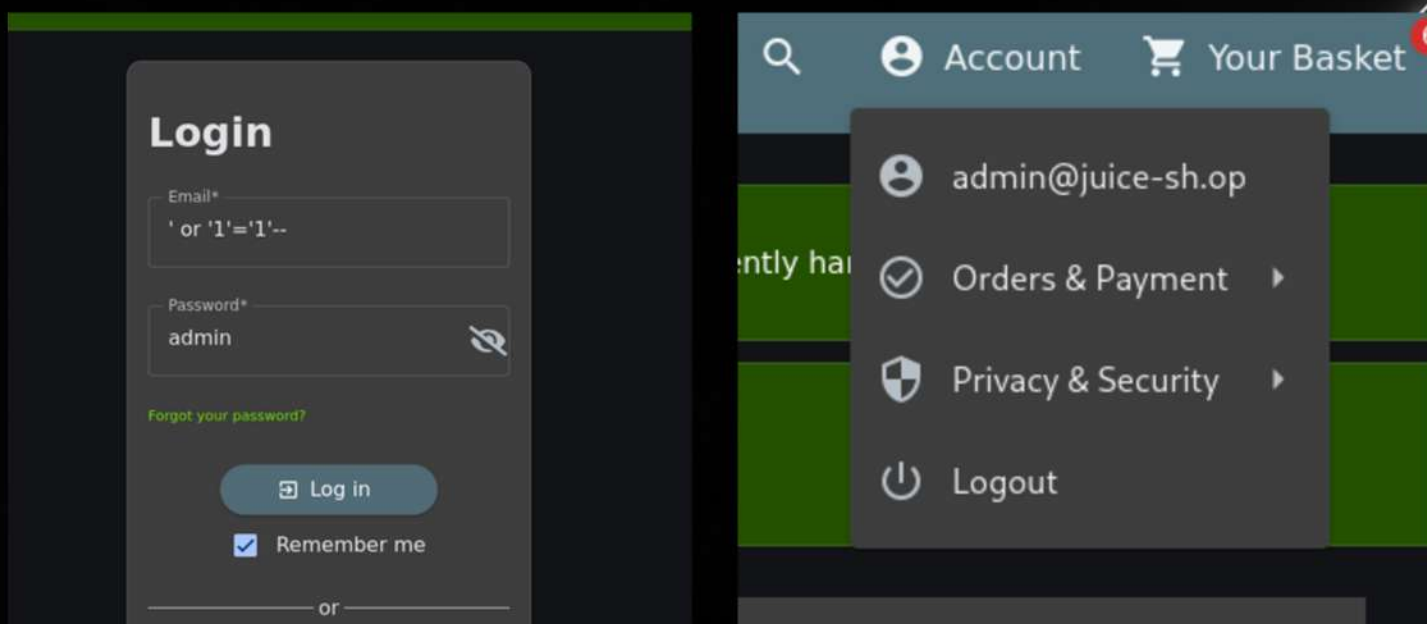
Findings Summary

ID	УЯЗВИМОСТЬ	OWASP TOP 10 2021	CVSS	Уровень	Статус
H-01	SQL Injection (Authentication Bypass)	A03:2021 Injection	9.8	Критический	NOT Fixed
H-02	Cryptographic Failures (JWT + MD5)	A02:2021 Cryptographic Failures	8.2	Высокий	NOT Fixed
H-03	Broken Access Control (API + подмена куки)	A01:2021 Broken Access Control	8.8	Высокий	NOT Fixed
M-01	DOM XSS (AngularJS)	A03:2021 Injection	6.1	Средний	NOT Fixed
L-01	Business Logic Abuse (удаление без проверки)	A01:2021 Broken Access Control	7.5	Высокий	NOT Fixed

DETAILED DESCRIPTION OF VULNERABILITIES

H-01: SQL Injection (Authentication Bypass) **Критический**

В форме аутентификации приложения отсутствует параметризация запросов. Пользовательский ввод напрямую подставляется в SQL-запрос к базе данных



Результат: Успешный обход аутентификации, получен доступ к административной учётной записи.

Рекомендации:

Использовать параметризованные запросы (Prepared Statements)

Внедрить WAF для защиты от типовых SQL-инъекций

Настроить централизованную обработку ошибок без вывода внутренней информации

H-02: Cryptographic Failures (JWT + MD5) **Высокий**

После успешного обхода аутентификации приложение передаёт JWT-токен, который содержит MD5-хэш пароля в открытом виде. Хэш был расшифрован с помощью hashcat.

Декодирован JWT-токен через jwt.io

Обнаружен MD5-хэш: 192023a7bbd73250516f069df18b500

Взломан через hashcat + rockyou.txt → admin123

```
{
  "status": "success",
  "data": {
    "id": 1,
    "username": "",
    "email": "admin@juice-sh.op",
    "password": "0192023a7bbd73250516f069df18b500",
    "role": "admin",
    "deluxeToken": "",
    "lastLoginIp": "",
    "profileImage": "assets/public/images/uploads/defaultAdmin.png",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2026-06-19 22:30:54.591 +00:00",
    "updatedAt": "2026-06-19 22:30:54.591 +00:00",
    "deletedAt": null
  },
  "iat": 1781977677
}
```

```
misfortune@misfortune:~$ hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-penryn-12th Gen Intel(R) Core(TM) i3-12100F, 4185/8370 MB (2048 MB allocatable), 5MCU

Min password length supported by kernel: 0
Maximum password length supported by kernel: 256
Most unbalanced: 12

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 513 MB (7385 MB free)

Dictionary cache hit:
* Filename ..: /usr/share/wordlists/rockyou.txt
* Passwords..: 14344385
* Bytes.....: 139921507
* Keyspace...: 14344385

0192023a7bbd73250516f069df18b500:admin123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 0192023a7bbd73250516f069df18b500
```

Результат: Получен пароль администратора в открытом виде, возможен повторный вход.

Причина: Ошибка реализации механизма аутентификации — bcrypt защищает хранение паролей в БД, но не предотвращает передачу хэша в JWT.

Рекомендации:

Не помещать чувствительные данные в JWT

(пароли, хэши, TOTP-секреты)

Использовать безопасные алгоритмы

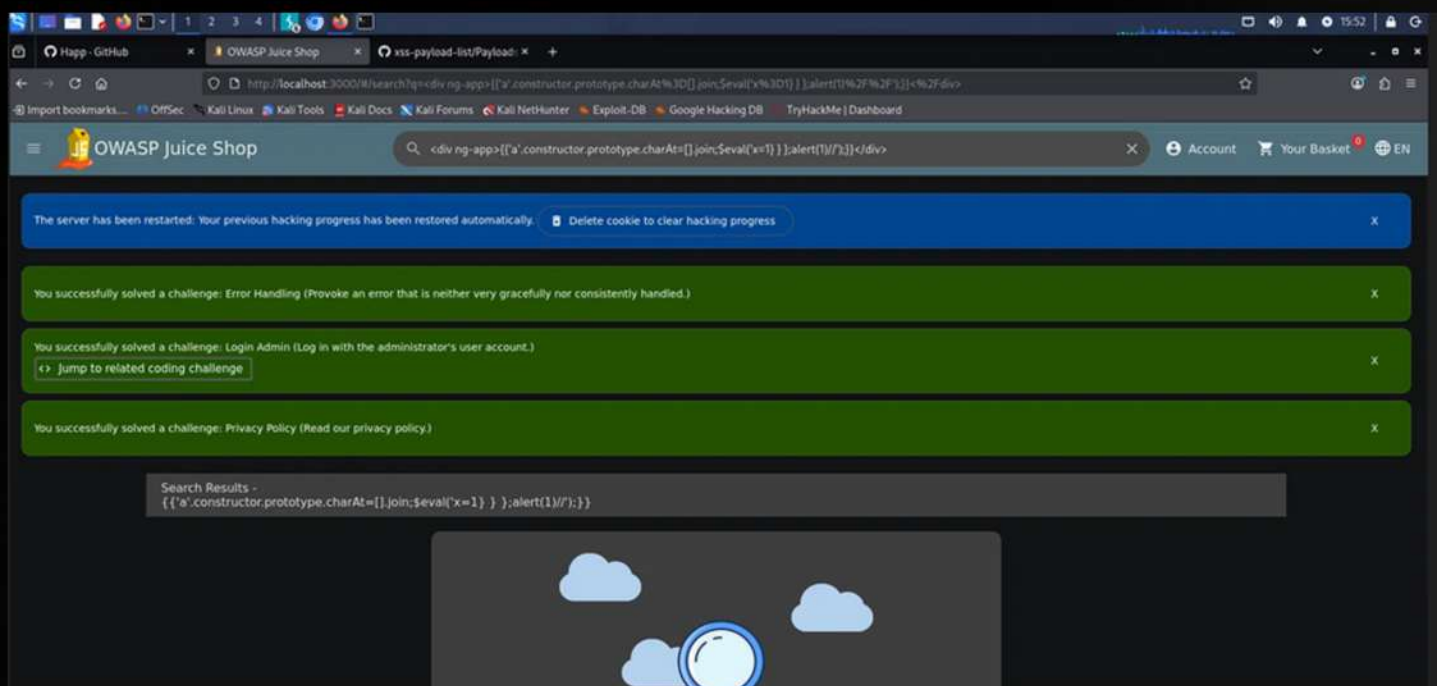
хеширования (Argon2, bcrypt, scrypt)

Передавать JWT исключительно по HTTPS (TLS)

Каждый запрос должен проверять принадлежность сессии

M-01: DOM XSS (AngularJS) Средний

В поле поиска обнаружена DOM-Based XSS через устаревший движок AngularJS. Сработала шаблонная инъекция с использованием синтаксиса `{{ }}` и вызовом `$eval`.



Результат: Выполнение произвольного JavaScript-кода в контексте веб-приложения.

Причина: Использование устаревшей версии AngularJS в гибридном приложении (Angular + AngularJS).

Рекомендации:

Обновить устаревшие версии фреймворков
Строгая фильтрация пользовательского ввода
Использовать WAF (Cloudflare WAF, AWS WAF, ModSecurity)

L-01: Business Logic Abuse (удаление user без проверки) **Высокий**

Функция Data Erasure Request не выполняет корректную проверку контрольного вопроса при удалении пользователя. Поле с контрольным вопросом принимает рандомное значение, после чего удаление выполняется успешно.

The screenshot shows a web interface for a 'Data Erasure Request (Art. 17 GDPR)'. The form includes a title, a paragraph about data security, and a section titled 'Request Data Erasure'. It contains two input fields: 'Confirm Email Address' with the value 'admin@juice-sh.op' and 'Answer' with the value 'aaaaa'. A button labeled 'Delete User Data' is positioned below the inputs. Below the form, a confirmation message states: 'Sorry to see you leave! Your erasure request will be processed shortly.' with a 'Go To HomePage' button.

Data Erasure Request (Art. 17 GDPR)

We take data security, customer privacy, and legal compliance very serious. In accordance with GDPR we allow you to request complete erasure of your account and any associated data.

Request Data Erasure

Confirm Email Address

admin@juice-sh.op

Answer

aaaaa

Delete User Data

Sorry to see you leave! Your erasure request will be processed shortly.

Go To HomePage

Результат: Несанкционированное удаление данных, нарушение целостности данных, обход предусмотренных механизмов подтверждения критической операции.

Рекомендации:

Обеспечить корректную серверную проверку ответа на контрольный вопрос
Использовать повторную аутентификацию для критических операций (ввод пароля, MFA, одноразовый код)

Fail-Safe: при некорректном ответе выполнение операции должно быть немедленно прекращено

Вести журнал аудита критических операций

ATTACK CHAIN SUMMARY

Reconnaissance



DOM XSS (AngularJS {{\$eval}})



SQL Injection (Authentication Bypass)



Admin Access (через SQLi)



Post Exploitation Enumeration



API Exploitation (GET /api/Users, PUT /api/Users/2)



JWT Analysis → MD5 Hash → hashcat → admin123



Broken Access Control (подмена куки)



Business Logic Abuse (удаление без проверки)



Full Application Compromise

ATTACK IMPACT

- Administrative Access Obtained
- Sensitive Data Exposure
- Credential Recovery
- Authentication Bypass
- Broken Access Control Exploitation
- API Exploitation (User Data Exposure, Role Modification)
- Business Logic Abuse
- Full Application Compromise

CONCLUSION

В ходе тестирования были выявлены критические уязвимости из категорий OWASP Top 10, которые позволили выполнить полный цикл компрометации веб-приложения от разведки до получения административного доступа и нарушения бизнес-логики системы.

Основные выводы:

Наиболее критической оказалась SQL Injection, позволившая обойти аутентификацию и получить доступ к учётной записи администратора.

JWT-токен содержит MD5-хэш пароля, что нарушает принципы криптографической безопасности и позволяет восстановить пароль.

Отсутствие контроля доступа на серверной стороне даёт возможность манипулировать данными через API.

Бизнес-логика содержит ошибки, позволяющие удалять пользователей без повторной аутентификации.

Рекомендованные меры:

Внедрение Prepared Statements для защиты от SQL-инъекций

Исключение чувствительных данных из JWT

Внедрение RBAC и проверки прав на каждом запросе

Обновление устаревших версий AngularJS

Повторная аутентификация для критических операций

Настройка WAF